

- Scenario:** A Pod is stuck in `ImagePullBackOff`.
Resolution: Check `kubectl describe pod` for "Auth" or "Not Found" errors and verify `imagePullSecrets`.
- Scenario:** A Pod is in `CrashLoopBackOff` with no error in the current logs.
Resolution: Run `kubectl logs <pod-name> --previous` to see the exit reason from the last crash.
- Scenario:** A Pod is `Pending` and the event says "0/3 nodes available: 3 Insufficient memory".
Resolution: Reduce the `resources.requests.memory` in the deployment or add a new node to the cluster.
- Scenario:** A Pod is in `CreateContainerConfigError`.
Resolution: Verify if the `ConfigMap` or `Secret` referenced in the YAML actually exists in the same namespace.
- Scenario:** A container exits with `OOMKilled` (Exit Code 137).
Resolution: Increase the `resources.limits.memory` or check the application code for memory leaks.
- Scenario:** A Pod is stuck in `Terminating` for a long time.
Resolution: Force delete using `kubectl delete pod <name> --force --grace-period=0`.
- Scenario:** A Service is created but the application is not reachable via the Service IP.
Resolution: Run `kubectl get ep` to check if the Endpoints are populated; if not, fix the selector labels.
- Scenario:** A Pod cannot resolve the DNS name of another service (e.g., `mysql-service`).
Resolution: Check the `coredns` pods in `kube-system` and ensure the service name and namespace are correct.
- Scenario:** An EBS Volume fails to attach with `Multi-Attach error`.
Resolution: Ensure the old Pod is fully deleted so the volume can detach from the old node and attach to the new one.

10. **Scenario:** A Pod shows `Permission Denied` when writing to a mounted volume.
Resolution: Update the `securityContext` with the correct `fsGroup` so the container user has write access.
11. **Scenario:** A Node shows `NotReady` status in the cluster.
Resolution: SSH into the node and run `systemctl restart kubelet` and check for disk or memory pressure.
12. **Scenario:** A `LivenessProbe` failure is causing the Pod to restart repeatedly.
Resolution: Increase the `initialDelaySeconds` or `failureThreshold` to give the app more time to start.
13. **Scenario:** A `ReadinessProbe` failure is keeping the Pod out of the LoadBalancer.
Resolution: Check application health endpoint logs to see why the dependency check is failing.
14. **Scenario:** A Deployment update is stuck and not progressing to new pods.
Resolution: Check `kubectl rollout status` and verify if the new image version exists in the registry.
15. **Scenario:** A Pod receives `InvalidImageName` error.
Resolution: Correct the image tag syntax in the deployment manifest (usually a typo in the registry URL).
16. **Scenario:** `FailedScheduling` error: "node(s) had taint {key: value}, that the pod didn't tolerate".
Resolution: Add the matching `tolerations` to the Pod spec or remove the taint from the node.
17. **Scenario:** A Pod is in `Error` state with Exit Code 1.
Resolution: Check application logs for "Database Connection Failed" or missing environment variables.
18. **Scenario:** `RunContainerError` occurs during pod startup.
Resolution: Check if the startup command/entrypoint in the YAML is valid and the file exists in the container.
19. **Scenario:** A LoadBalancer Service stays in `<pending>` state for external IP.
Resolution: Ensure the Cloud Controller Manager is running and you haven't hit the cloud provider's LB quota.

20. **Scenario:** `ImagePullBackOff` due to "too many requests" from DockerHub.
Resolution: Configure a private registry mirror or add an authenticated DockerHub account to `imagePullSecrets`.
21. **Scenario:** A Pod can talk to the Internet but cannot reach other Pods in the cluster.
Resolution: Check `NetworkPolicies` to see if ingress/egress rules are blocking internal traffic.
22. **Scenario:** PVC is stuck in `Pending` with "storageclass not found".
Resolution: Create the missing `StorageClass` or update the PVC to use an existing one like `gp2` or `standard`.
23. **Scenario:** A Node is under `DiskPressure` and evicting pods.
Resolution: Run `docker image prune -a` on the node or expand the root volume size in AWS/Azure.
24. **Scenario:** `kubectl exec` fails with "error: sidecar container not found".
Resolution: Specify the container name using `-c <container-name>` if the pod has multiple containers.
25. **Scenario:** `ConfigMap` updates are not reflecting in the application.
Resolution: Restart the Pods (rollout restart) as env variables from ConfigMaps are not dynamic.
26. **Scenario:** A Pod cannot access the K8s API (403 Forbidden).
Resolution: Check the `ServiceAccount` and ensure a `RoleBinding` or `ClusterRoleBinding` is attached.
27. **Scenario:** `502 Bad Gateway` error appearing via Nginx Ingress.
Resolution: Ensure the backend Service port in the Ingress YAML matches the actual Service port.
28. **Scenario:** A `StatefulSet` pod is stuck because the volume is in a different Availability Zone.
Resolution: Use `volumeBindingMode: WaitForFirstConsumer` in the `StorageClass` to align volume and node zones.

29. **Scenario:** `Kubelet` is failing to start on a new worker node.
Resolution: Check `journalctl -u kubelet` and ensure the swap memory is disabled (`swapoff -a`).
30. **Scenario:** A Pod is in `ImagePullBackOff` specifically on one node only.
Resolution: Check the node's network connectivity to the registry or check for disk space issues on that node.
31. **Scenario:** `CPU Throttling` is causing the application to run very slowly.
Resolution: Increase the `resources.limits.cpu` or remove the limit to allow the pod to burst.
32. **Scenario:** `HorizontalPodAutoscaler` (HPA) is not scaling the pods.
Resolution: Ensure `resources.requests` are defined for the pods; HPA needs them to calculate percentages.
33. **Scenario:** A Pod receives `Exit Code 143`.
Resolution: This is a graceful `SIGTERM`; check if a deployment update or a `drain` command triggered it.
34. **Scenario:** `413 Request Entity Too Large` error via Ingress.
Resolution: Add the annotation `nginx.ingress.kubernetes.io/proxy-body-size: "50m"` to the Ingress resource.
35. **Scenario:** `MountVolume.SetUp failed` for a Secret.
Resolution: Check if the Secret name in the `volumes` section of the Pod matches the actual Secret name.
36. **Scenario:** `Error from server (Forbidden)` when running `kubectl get nodes`.
Resolution: Update your `kubeconfig` or check if your IAM/RBAC permissions have expired.
37. **Scenario:** `FailedCreate` event in a `ReplicaSet`.
Resolution: Check if you have reached the Resource Quota limits for the namespace.
38. **Scenario:** Pods are being killed frequently with no OOM error.
Resolution: Check if `Preemption` is happening because a higher-priority pod needs the space.

39. **Scenario:** `Ingress` controller is not picking up new rules.
Resolution: Check the Ingress controller logs for syntax errors in your Ingress YAML.
40. **Scenario:** A `CronJob` is not starting any new jobs.
Resolution: Check `concurrencyPolicy` and ensure the previous job hasn't hung in a "Running" state.
41. **Scenario:** `MatchNodeSelector` failed for a pod.
Resolution: Verify that the node labels match the `nodeSelector` key-value pairs in the Pod spec.
42. **Scenario:** `Pod Security Policy` (PSP) is blocking pod creation.
Resolution: Ensure the ServiceAccount has the `use` permission for the required PSP.
43. **Scenario:** `Out of Pods` error on an AWS EKS node.
Resolution: Check the ENI limits for the instance type; you may need to use a larger instance.
44. **Scenario:** `Connection Timeout` when a pod tries to reach a public API.
Resolution: Check if the NAT Gateway is working or if the Security Group allows outbound traffic on port 443.
45. **Scenario:** `Error: failed to create containerd task`.
Resolution: This usually indicates the node is out of PIDs or memory; restart the node or containerd.
46. **Scenario:** `Helm install` fails with "cannot re-use a name that is still in use".
Resolution: Run `helm list -A` to find the existing release or use `helm upgrade --install`.
47. **Scenario:** `Kube-proxy` is failing to sync iptables.
Resolution: Check if the node has the `ip_tables` kernel module loaded and `conntrack` is installed.
48. **Scenario:** `Namespace` stuck in `Terminating` state.
Resolution: Check for resources with finalizers (like `apiservices`) that are blocking the deletion.

49. **Scenario:** `Too many open files` error in container logs.

Resolution: Increase the `ulimit` in the container's securityContext or on the host node.

50. **Scenario:** `ETCD` is running slow, causing `kubectl` timeouts.

Resolution: Check the disk IOPS on the master nodes; etcd requires high-performance SSDs.

51. **Scenario:** A Pod is in `ImagePullBackOff` due to "denied: requested access to the resource is denied".

Resolution: Verify that the `imagePullSecrets` name in the Pod YAML exactly matches the Secret created in that namespace.

52. **Scenario:** `CrashLoopBackOff` occurs only when the application tries to connect to the database.

Resolution: Check the `ConfigMap` or `Secret` environment variables for correct DB Host, Port, and Credentials.

53. **Scenario:** A Pod is `Pending` with the message "node(s) had untolerated taint {node.kubernetes.io/unschedulable: }".

Resolution: The node is marked as unschedulable (cordoned); run `kubectl uncordon <node-name>` to allow scheduling.

54. **Scenario:** `CreateContainerError` due to "container name 'x' is invalid".

Resolution: Ensure container names follow DNS-1123 standards (lowercase alphanumeric characters or '-' only).

55. **Scenario:** A container exits with `Exit Code 143`.

Resolution: This is a SIGTERM; check if the `HorizontalPodAutoscaler` is scaling down or if a `rollout` is in progress.

56. **Scenario:** `FailedMount` error: "mount failed: exit status 32" for an NFS share.

Resolution: Install the `nfs-utils` (or `nfs-common`) package on the worker node to support NFS mounting.

57. **Scenario:** A Service of type `LoadBalancer` is working, but `External-IP` is showing `<pending>` for hours.

Resolution: Verify if your Cloud Provider (AWS/Azure) has reachable subnets and that the IAM role has permission to create LBs.

58. **Scenario:** `503 Service Unavailable` error when accessing the app through an Ingress.

Resolution: Check if the Pods are passing their `ReadinessProbe`; if they aren't, the Service has no active endpoints.

59. **Scenario:** A Pod is killed with `Exit Code 139` (Segmentation Fault).

Resolution: This is a binary/memory error; check for incompatible library versions inside the Docker image.

60. **Scenario:** `Error from server (Timeout)` when running any `kubectl` command.

Resolution: Check if the `kube-apiserver` is running and ensure your local machine has network access to the API endpoint.

61. **Scenario:** `FailedScheduling` error: "nodes are available: 3 node(s) didn't match pod anti-affinity rules".

Resolution: Check your `podAntiAffinity` settings; you may be trying to run more replicas than you have available nodes.

62. **Scenario:** A `Secret` was updated, but the Pod using it as an environment variable still has the old value.

Resolution: Environment variables are injected at startup; perform a `kubectl rollout restart` to pick up the new Secret values.

63. **Scenario:** `FailedPostStartHook` error in Pod events.

Resolution: The script or command defined in the `postStart` handler failed; check the command syntax and container paths.

64. **Scenario:** `NetworkPlugin cni failed to set up pod network` error.

Resolution: Check the logs of your CNI provider (Calico/Weave/AWS-VPC-CNI) pods in the `kube-system` namespace.

65. **Scenario:** `Namespace` stuck in `Terminating` with "DiscoveryAddress" errors.

Resolution: Manually edit the namespace YAML and remove the entries under the `finalizers` section to force deletion.

66. **Scenario:** `FailedCreate` error in a StatefulSet: "pod has unbound immediate PersistentVolumeClaims".

Resolution: Ensure the `volumeClaimTemplates` match an available `StorageClass` that can provision volumes in that zone.

67. **Scenario:** A Pod can ping an IP address but cannot resolve any domain names (DNS failure).

Resolution: Check `/etc/resolv.conf` inside the pod and ensure the `kube-dns` or `coredns` service is running.

68. **Scenario:** `404 Not Found` when accessing a specific path via Ingress.

Resolution: Verify the `path` and `pathType` (Prefix/Exact) in the Ingress rules match your application's routing.

69. **Scenario:** `PreStartContainer` error appearing in events.

Resolution: This usually relates to a failure in a `privileged` container or an `AppArmor` profile blocking the startup.

70. **Scenario:** `Back-off restarting failed container`.

Resolution: This is a generic K8s wait period; look at the **previous** event in `kubectl describe` to find the real error.

71. **Scenario:** `DiskPressure` taint is appearing on nodes even though there is 20% free space.

Resolution: Check the `evictionHard` settings in the `Kubelet` configuration; the threshold might be set too high.

72. **Scenario:** `MatchNodeSelector` failed for a specific GPU workload.

Resolution: Ensure the worker node has the correct GPU labels (e.g., `accelerator=nvidia`) applied manually or by a plugin.

73. **Scenario:** `FailedAttachVolume` error: "Volume is already exclusively attached to one node".

Resolution: This happens with RWO volumes; ensure the previous pod on a different node is completely terminated.

74. **Scenario:** `Kube-proxy` is consuming 100% CPU on a node.

Resolution: This often happens with thousands of Services; switch `kube-proxy` mode from `iptables` to `ipvs`.

75. **Scenario:** `Error: container's ID contains invalid characters`.

Resolution: This usually indicates a conflict between the Container Runtime (Docker/containerd) and the Kubelet; restart both.

76. **Scenario:** `LivenessProbe` fails only when the application is under heavy load.
Resolution: Increase the `timeoutSeconds` or `failureThreshold` to account for slower response times during peak traffic.

77. **Scenario:** `FailedScheduling` error: "nodes are available: 3 node(s) had volume node affinity conflict".
Resolution: The Pod is being scheduled on a node in Zone A, but the existing Persistent Volume is in Zone B.

78. **Scenario:** `Too many open files` error on the host node, not just the container.
Resolution: Increase the `fs.file-max` parameter in the node's `/etc/sysctl.conf`.

79. **Scenario:** `Admission Webhook` error preventing any new Pods from being created.
Resolution: Check the logs of the webhook server (e.g., Istio, Linkerd, or a custom policy engine) to see why it's rejecting requests.

80. **Scenario:** `Invalid value: "v1": versions must match the format "v[0-9]+"` in a YAML file.
Resolution: Check the `apiVersion` string; you likely have a typo or are using an API version that is deprecated.

81. **Scenario:** `Kubelet` logs show "Failed to get system container stats".
Resolution: Ensure `cgroups` is properly configured on the Linux host and matches the `cgroupDriver` in Kubelet config.

82. **Scenario:** `ConfigMap` data is showing up as garbled text or binary in the container.
Resolution: Ensure the `ConfigMap` was created using `--from-file` correctly and isn't being interpreted as a binary blob.

83. **Scenario:** `FailedBinding` for a PVC: "requested spec.storageClassName does not exist".
Resolution: Run `kubectl get sc` to see available storage classes and update your PVC YAML accordingly.

84. **Scenario:** `NodeNotReady` due to `NetworkUnavailable`.
Resolution: The CNI hasn't assigned an IP range to the node; check if the `kube-controller-manager` has the `--allocate-node-cidrs` flag set.

85. **Scenario:** `kubectl log` returns "Error from server: Get ...: dial tcp ... i/o timeout".

Resolution: There is a network block between the Master and Worker node on port 10250 (Kubelet port).

86. **Scenario:** `FailedToUpdateEndpoint` errors in the `kube-controller-manager` logs.

Resolution: This can happen if the `apiserver` is overloaded; check etcd performance and master node CPU.

87. **Scenario:** `Forbidden` error when a Pod tries to list other Pods in the namespace.

Resolution: Create a `Role` with `get`, `list` permissions for pods and bind it to the Pod's `ServiceAccount`.

88. **Scenario:** `ImagePullBackOff` for a public image on a private cluster with no NAT Gateway.

Resolution: Set up a **Private Link** or **VPC Endpoint** for the Container Registry to allow internal traffic.

89. **Scenario:** `HorizontalPodAutoscaler` is flapping (repeatedly scaling up and down).

Resolution: Adjust the `stabilizationWindowSeconds` in the HPA spec to slow down downscaling.

90. **Scenario:** `Error: failed to reserve container name`.

Resolution: A previous container with the same name is still being cleaned up by the runtime; wait or restart the runtime.

91. **Scenario:** `FailedScheduling` error: "1 Insufficient pods".

Resolution: The node has reached its maximum allowed pods (default 110); you must add more nodes.

92. **Scenario:** `Warning: Certificate Expired` in the cluster components.

Resolution: Run `kubeadm certs renew all` (if using kubeadm) and restart the control plane components.

93. **Scenario:** `Back-off pulling image` even though the credentials are correct.

Resolution: Check if the node has reached its storage limit for images; old images may need to be pruned.

94. **Scenario:** `Error: failed to start container: error linking ...: file exists.`

Resolution: Clean up the `/var/lib/containerd` (or `docker`) storage directory as it has corrupted metadata.

95. **Scenario:** `TaintManagerEviction` event in pod logs.

Resolution: The pod was evicted because a new taint was added to the node or the node went `NotReady`.

96. **Scenario:** `Service` of type `NodePort` is not accessible on the node's IP.

Resolution: Check the node's **Security Group** or Firewall to ensure the port (30000-32767) is open.

97. **Scenario:** `ResourceQuota` exceeded for a namespace.

Resolution: Delete unused resources or request an increase in the `ResourceQuota` object for that namespace.

98. **Scenario:** `Ingress` controller shows `Empty address` in `kubectl get ing`.

Resolution: Ensure the Ingress controller is actually running and has been assigned a LoadBalancer IP.

99. **Scenario:** Pod is stuck in `ContainerCreating` for a long time without errors.

Resolution: This often indicates a slow volume mount or a slow CNI IP assignment; check `describe pod` for specific "Mounting" events.

100. **Scenario:** `ETCD` logs show "request ignored (superseded)".

Resolution: This indicates high leader election frequency; check network stability between master nodes.